GB EN · BD BN · Saved

DEVELOPER REFERENCE

Git & GitHub Cheat Sheet

A concise, hands-on reference for every Git command — with real examples, GitHub workflow, and recovery techniques all in one place.

$ every command · with examples · explained simply

Search commands... ⌘K

// What is Git?

Version Control System
Tracks every change to your code over time, like a time machine for files.

Branching & Merging
Work on features in isolation, then combine them safely.

Collaboration
Multiple developers work on the same project without conflicts.

GitHub = Cloud + Collaboration
GitHub hosts your Git repos online. Push code, open PRs, review changes.

01 — SETUP & INITIALIZE

Global Config
Set your identity once

Set username & email

```
git config --global user.name "John Doe"
git config --global user.email "john@email.com"
```

Set default editor & branch name

```
git config --global core.editor "code"
git config --global init.defaultBranch main
```

View all config

```
git config --list
```

Init & Clone
Start a repo locally or from remote

Create new local repo

```
git init my-project
cd my-project
```

Clone from GitHub

```
git clone https://github.com/user/repo.git
```

Clone into a specific folder

```
git clone https://github.com/user/repo.git my-folder
```

02 — STAGING & COMMITS

Staging Files
Choose what goes into the next commit

Check current status

```
git status
```

Stage a specific file

```
git add index.html
```

Stage all changed files

```
git add .
```

Unstage a file

```
git restore --staged index.html
```

See what's staged vs unstaged

```
git diff --staged
```

Committing
Save a snapshot of staged changes

Commit with a message

```
git commit -m "feat: add login page"
```

Stage + commit tracked files at once

```
git commit -am "fix: typo in header"
```

Amend last commit message

```
git commit --amend -m "fix: correct message"
```

Empty commit (trigger CI)

```
git commit --allow-empty -m "chore: trigger CI"
```

03 — BRANCHING

Branch Basics
Create, switch, list, delete

List all branches

```
git branch            # local
git branch -a         # local + remote
```

Create a new branch

```
git branch feature/login
```

Switch to a branch

```
git switch feature/login
```

Create & switch in one step

```
git switch -c feature/signup
```

Delete a branch

```
git branch -d feature/login   # safe
git branch -D feature/login   # force
```

Rename current branch

```
git branch -m new-name
```

See it in action →

Merge & Rebase
Combine branch histories

Merge a branch into current

```
git switch main
git merge feature/login
```

Merge without fast-forward (keeps history)

```
git merge --no-ff feature/login
```

Rebase onto main (cleaner history)

```
git switch feature/login
git rebase main
```

Abort a conflicting rebase

```
git rebase --abort
```

Continue after resolving conflicts

```
git rebase --continue
```

See it in action →

04 — REMOTE & GITHUB

Remote Commands
Connect & sync with GitHub

Add a remote (GitHub)

```
git remote add origin https://github.com/user/repo.git
```

View remotes

```
git remote -v
```

Push branch to GitHub

```
git push origin main
```

Push & set upstream (first time)

```
git push -u origin feature/login
```

Pull latest from remote

```
git pull origin main
```

Fetch (download without merge)

```
git fetch origin
```

Delete a remote branch

```
git push origin --delete feature/login
```

GitHub Workflow
Fork → PR → Review → Merge

1. **Fork & clone repo from GitHub**

```
git clone https://github.com/you/forked-repo.git
```

2. **Create a feature branch**

```
git switch -c feature/my-change
```

3. **Commit your changes**

```
git add .
git commit -m "feat: my change"
```

4. **Push to your fork**

```
git push -u origin feature/my-change
```

5. **Open Pull Request on GitHub UI**

```
# Go to GitHub → Compare & Pull Request
# Add title, description → Submit PR
```

6. **Sync with upstream after merge**

```
git switch main
git pull upstream main
```

05 — UNDO & RECOVERY

Undo Changes
Revert, reset, restore

Discard changes in working directory

```
git restore index.html
```

Undo last commit (keep changes)

```
git reset --soft HEAD~1
```

Undo last commit (discard changes) ⚠️

```
git reset --hard HEAD~1
```

Revert a commit (safe, creates new commit)

```
git revert a3f1b2c
```

Recover lost commits with reflog

```
git reflog
git reset --hard HEAD@{3}
```

See it in action →

Stash
Save work temporarily without committing

Stash current changes

```
git stash
```

Stash with a label

```
git stash push -m "WIP: login form"
```

List all stashes

```
git stash list
```

Apply latest stash

```
git stash pop
```

Apply a specific stash

```
git stash apply stash@{2}
```

Drop a stash

```
git stash drop stash@{0}
```

See it in action →

06 — LOG & HISTORY

git log
Inspect commit history

Basic log

```
git log
```

Compact one-line log

```
git log --oneline
```

Visual branch graph

```
git log --oneline --graph --all
```

Filter by author

```
git log --author="Alice"
```

Log since a date

```
git log --since="2026-01-01"
```

Reflog (every HEAD movement)

```
git reflog
```

Tags
Mark release versions

List all tags

```
git tag
```

Create a lightweight tag

```
git tag v1.0.0
```

Create an annotated tag

```
git tag -a v1.0.0 -m "Release v1.0.0"
```

Push tags to GitHub

```
git push origin --tags
```

Delete a tag

```
git tag -d v1.0.0
git push origin --delete v1.0.0
```

07 — ADVANCED COMMANDS

Cherry-pick
Copy specific commits to current branch

Apply a single commit from another branch

```
git cherry-pick a3f1b2c
```

Cherry-pick a range of commits

```
git cherry-pick a3f1b2c..f9e3d11
```

Cherry-pick without committing

```
git cherry-pick -n a3f1b2c
```

See it in action →

Diff & Blame
Inspect what changed and who changed it

See unstaged changes

```
git diff
```

Compare two branches

```
git diff main..feature/login
```

See who changed each line

```
git blame index.html
```

Search for a string in history

```
git log -S "loginUser" --oneline
```

Interactive Rebase
Rewrite, squash, edit commits

Squash last 3 commits interactively

```
git rebase -i HEAD~3
```

In the editor that opens:

```
pick a3f1b2c feat: add login
squash 9d4e521 fix: typo
squash 1bc7830 fix: spacing
# → becomes 1 clean commit
```

See it in action →

.gitignore
Tell Git what to never track

Example .gitignore file

```
# Dependencies
node_modules/
# Environment secrets
.env
.env.local
# Build output
dist/
build/
# OS files
.DS_Store
Thumbs.db
```

Ignore already-tracked file

```
git rm --cached .env
```

08 — QUICK REFERENCE

COMMAND	WHAT IT DOES
`git status`	Show working tree status
`git add .`	Stage all changes
`git commit -m "msg"`	Commit with message
`git push origin main`	Push to GitHub
`git pull origin main`	Pull latest from GitHub
`git switch -c branch`	Create + switch to branch
`git merge branch`	Merge branch into current
`git stash`	Save changes temporarily
`git stash pop`	Restore stashed changes
`git log --oneline`	Compact commit history
`git reflog`	Every HEAD movement
`git reset --soft HEAD~1`	Undo last commit, keep changes
`git reset --hard HEAD~1`	Undo last commit, discard changes
`git revert abc123`	Safe undo via new commit
`git cherry-pick abc123`	Copy commit to current branch
`git tag v1.0.0`	Create a version tag
`git diff main..branch`	Compare two branches
`git blame file.txt`	See who wrote each line
`git rebase -i HEAD~3`	Squash/edit last 3 commits
`git remote -v`	List remotes with URLs